Predicting Electricity Demand Based on Weather

April 19, 2026

Group 5 Members:

- Behnam Taghavi-Fard
- Margaret Peacock
- Matthew Yeung
- Srinivas R Sudala

Problem Statement:

The SERC Reliability Corporation's 2025–2035 Regional Reliability Assessment found that nearly half of the SERC region failed to meet reliability criteria across the entire study period, placing these areas at elevated or high risk of being unable to meet future electricity demand (SERC, 2026). The SERC region covers the southeastern and central United States, spanning from Virginia and the Carolinas through Florida and west into the Mississippi and Missouri valleys.

Understanding what drives demand above expected levels is central to addressing these resource adequacy challenges. Weather is a natural starting point, particularly in the SERC region where extreme summer heat and winter cold events routinely push electricity consumption well beyond typical levels. This project develops two predictive models to estimate electricity demand based on weather conditions, with a focus on temperature and precipitation as key drivers. The goal is to develop and compare the performance of two separate models to find which one best fits this use case.

Data Identification:

For energy usage estimation, the U.S. Energy Information Administration (EIA) provides publicly available grid operations and retail electricity data suitable for this project. The two most relevant datasets are EIA Form 930 and EIA Form 861:

- EIA 930: Form EIA-930 collects hourly demand by region.
- EIA 861 M: Form EIA-861 M is a monthly census of all U.S. electric utilities, collecting data on retail electricity sales, revenue, and customer counts.

For weather information, National Oceanic and Atmospheric Administration (NOAA) provides ISD-Lite, which is a simplified version of NOAA's Integrated Surface Database that provides hourly weather observations from stations across the globe. Each record contains eight variables including air temperature, dew point, sea level pressure, wind direction and speed, cloud cover, and precipitation.

The data above is publicly available for use, without any Intellectual Property (IP) obstacles.

Data Availability:

Energy Demand Data Source Option 1: EIA 930 - Hourly Demand by Sub-Region: EIA form 930 provides electricity demand information by sub-region, date, and time.The EIA provides a free API with access to the form 930 data (API Dashboard - U.S. Energy Information Administration (EIA)). This allows us to directly query the dataset for the information that we need. The data can also be downloaded from their website. Core fields included in this file are:

- Period (date/time of the report)
- Subregion and subregion abbreviation
- Parent region

- Value (for demand)
- Value unit (megawatt hours)

Energy Demand Data Source Option 2: EIA 930 - Hourly Demand, Forecast, Generation and Interchange by Region: Similar to 930: Hourly Demand by Subregion, but only at parent region level and with additional features such as demand forecast and net generation broken down by source. Data is available as far back as 2015, and can be downloaded as csv files from www.eia.gov/electricity/gridmonitor/dashboard/electric_overview/US48/US48. These files include over 40 fields, but the most relevant fields to our project are:

- Region
- Date/time
- Demand
- Net Generation
- Total Interchange
- Adjusted demand
- Adjusted net generation
- Adjusted total interchange

Energy Demand Data Source Option 3: EIA 861 M: An alternative data source would be to use the EIA 860/861M. These forms provide retail sales of electricity to end use customers by sector and state. This information is available in xlsx format for 1991 - 2025, and can be downloaded from the EIA website: `https://www.eia.gov/electricity/data/eia861m/`. EIA 861 (Sales and Revenue) can be downloaded in xlsx format from Form EIA-861M (formerly EIA-826) detailed data - U.S. Energy Information Administration (EIA). Main features include:

- Year/month
- State
- Revenue (broken down by Residential, Industrial, Commercial, Transportation)
- Sales (broken down by Residential, Industrial, Commercial, Transportation)
- Customers (broken down by Residential, Industrial, Commercial, Transportation)

Weather Information: For weather information, National Oceanic and Atmospheric Administration (NOAA) provides ISD-Lite, which is a simplified version of NOAA's Integrated Surface Database that provides hourly weather observations from stations across the globe. Each record contains eight variables including air temperature, dew point, sea level pressure, wind direction and speed, cloud cover, and precipitation. Data is available from 1901 through Aug 2025, and can be downloaded as a compressed collection of csv files from NOAA ISD-Lite website. The website includes technical documentation on how to use the files, since they are fixed length with no headers. Core features include:

- Station code (in the file name. Used to identify location)
- Date/time
- Air temperature
- Dew point
- Sea level pressure
- Wind direction & speed
- Sky condition total coverage code
- Liquid precipitation depth (one-hour and six-hour)

Data Suitability:

Option 1: EIA 930 - Hourly Demand by Subregion: Using the API, and after requesting a free API key from the EIA, we can easily download hourly demand for any desired sub-region and timeframe. Example:

[3]:
```
import requests
import pandas as pd
url = "https://api.eia.gov/v2/electricity/rto/region-sub-ba-data/data/"
all_data = []
offset = 0
while True:
    params = {
        "api_key": "",
```

```
        "frequency": "hourly", "data[0]": "value","facets[subba][]": "4006",
        "start": "2023-01-01T00","end": "2026-01-01T00","sort[0][column]": "period",
        "sort[0][direction]": "asc","offset": offset,"length": 5000
    }
    response = requests.get(url, params=params)
    data = response.json()
    rows = data["response"]["data"]
    if not rows:
        break
    all_data.extend(rows)
    offset += 5000
df_EIA_930 = pd.DataFrame(all_data)

#change data types for value and period:
df_EIA_930["value"] = df_EIA_930["value"].astype("float")
df_EIA_930["period"] = pd.to_datetime(df_EIA_930["period"])
df_EIA_930.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 26305 entries, 0 to 26304
Data columns (total 7 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   period       26305 non-null  datetime64[ns]
 1   subba        26305 non-null  object
 2   subba-name   26305 non-null  object
 3   parent       26305 non-null  object
 4   parent-name  26305 non-null  object
 5   value        26305 non-null  float64
 6   value-units  26305 non-null  object
dtypes: datetime64[ns](1), float64(1), object(5)
memory usage: 1.4+ MB
```

[13]:
```
print(f"Min period: {min(df_EIA_930["period"])}, max period: {max(df_EIA_930["period"])}")
print(f"Min value: {min(df_EIA_930["value"])}, max value: {max(df_EIA_930["value"])}")
```

```
Min period: 2023-01-01 00:00:00, max period: 2026-01-01 00:00:00
Min value: 818.0, max value: 3615.0
```

Based on the date/time and values, data seems suitable for machine learning.

Option 2: EIA 930 - Hourly Demand, Forecast, Generation, and Interchange: This information is available to download as csv from the EIA website. Example:

[111]:
```
df_eia_930_balance = pd.read_csv("EIA/EIA 930/EIA930_BALANCE_2024_Jul_Dec.csv")
#Change date to appropriate data type:
df_eia_930_balance["Data Date"] = pd.to_datetime(df_eia_930_balance["Data Date"])
df_eia_930_balance.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 269426 entries, 0 to 269425
Data columns (total 65 columns):
 #   Column                                                                  Non-Null Count   Dtype
---  ------                                                                  --------------   -----
 0   Balancing Authority                                                     269426 non-null  object
 1   Data Date                                                               269426 non-null  datetime64[ns]
 2   Hour Number                                                             269426 non-null  int64
 3   Local Time at End of Hour                                               269426 non-null  object
 4   UTC Time at End of Hour                                                 269426 non-null  object
 5   Demand Forecast (MW)                                                    233476 non-null  float64
 6   Demand (MW)                                                             233836 non-null  float64
 7   Net Generation (MW)                                                     269140 non-null  float64
 8   Total Interchange (MW)                                                  267223 non-null  float64
```

```
 9   Sum(Valid DIBAs) (MW)                                          268473 non-null  float64
 10  Demand (MW) (Imputed)                                          304 non-null     float64
 11  Net Generation (MW) (Imputed)                                  360 non-null     float64
 12  Total Interchange (MW) (Imputed)                               6 non-null       float64
 13  Demand (MW) (Adjusted)                                         234066 non-null  float64
 14  Net Generation (MW) (Adjusted)                                 269375 non-null  float64
 15  Total Interchange (MW) (Adjusted)                              267223 non-null  float64
 16  Net Generation (MW) from Coal                                  153344 non-null  float64
 17  Net Generation (MW) from Natural Gas                           220529 non-null  float64
 18  Net Generation (MW) from Nuclear                               79321 non-null   float64
 19  Net Generation (MW) from All Petroleum Products                92069 non-null   float64
 20  Net Generation (MW) from Hydropower Excluding Pumped Storage   196944 non-null  float64
 21  Net Generation (MW) from Pumped Storage                        1392 non-null    float64
 22  Net Generation (MW) from Solar without Integrated Battery Storage  189620 non-null  float64
 23  Net Generation (MW) from Solar with Integrated Battery Storage  5832 non-null    float64
 24  Net Generation (MW) from Wind without Integrated Battery Storage  141180 non-null  float64
 25  Net Generation (MW) from Wind with Integrated Battery Storage  0 non-null       float64
 26  Net Generation (MW) from Battery Storage                       6555 non-null    float64
 27  Net Generation (MW) from Other Energy Storage                  96 non-null      float64
 28  Net Generation (MW) from Unknown Energy Storage                1681 non-null    float64
 29  Net Generation (MW) from Geothermal                            144 non-null     float64
 30  Net Generation (MW) from Other Fuel Sources                    158747 non-null  float64
 31  Net Generation (MW) from Unknown Fuel Sources                  0 non-null       float64
 32  Net Generation (MW) from Coal (Imputed)                        186 non-null     float64
 33  Net Generation (MW) from Natural Gas (Imputed)                 283 non-null     float64
 34  Net Generation (MW) from Nuclear (Imputed)                     155 non-null     float64
 35  Net Generation (MW) from All Petroleum Products (Imputed)      132 non-null     float64
 36  Net Generation (MW) from Hydropower Excluding Pumped Storage (Imputed)  231 non-null  float64
 37  Net Generation (MW) from Pumped Storage (Imputed)              0 non-null       float64
 38  Net Generation (MW) from Solar without Integrated Battery Storage (Imputed)  284 non-null  float64
 39  Net Generation (MW) from Solar with Integrated Battery Storage (Imputed)  0 non-null  float64
 40  Net Generation (MW) from Wind without Integrated Battery Storage (Imputed)  133 non-null  float64
 41  Net Generation (MW) from Wind with Integrated Battery Storage (Imputed)  0 non-null  float64
 42  Net Generation (MW) from Battery Storage (Imputed)             0 non-null       float64
 43  Net Generation (MW) from Other Energy Storage (Imputed)        0 non-null       float64
 44  Net Generation (MW) from Unknown Energy Storage (Imputed)      0 non-null       float64
 45  Net Generation (MW) from Geothermal (Imputed)                  0 non-null       float64
 46  Net Generation (MW) from Other Fuel Sources (Imputed)          1649 non-null    float64
 47  Net Generation (MW) from Unknown Fuel Sources (Imputed)        0 non-null       float64
 48  Net Generation (MW) from Coal (Adjusted)                       153529 non-null  float64
 49  Net Generation (MW) from Natural Gas (Adjusted)                220812 non-null  float64
 50  Net Generation (MW) from Nuclear (Adjusted)                    79476 non-null   float64
 51  Net Generation (MW) from All Petroleum Products (Adjusted)     92201 non-null   float64
 52  Net Generation (MW) from Hydropower Excluding Pumped Storage (Adjusted)  197146 non-null  float64
 53  Net Generation (MW) from Pumped Storage  (Adjusted)            1392 non-null    float64
 54  Net Generation (MW) from Solar without Integrated Battery Storage (Adjusted)  189896 non-null  float64
 55  Net Generation (MW) from Solar witho Integrated Battery Storage (Adjusted)  5832 non-null  float64
 56  Net Generation (MW) from Wind without Integrated Battery Storage (Adjusted)  141311 non-null  float64
 57  Net Generation (MW) from Wind with Integrated Battery Storage (Adjusted)  0 non-null  float64
 58  Net Generation (MW) from Battery Storage (Adjusted)            6555 non-null    float64
 59  Net Generation (MW) from Other Energy Storage (Adjusted)       96 non-null      float64
 60  Net Generation (MW) from Unknown Energy Storage (Adjusted)     1681 non-null    float64
 61  Net Generation (MW) from Geothermal (Adjusted)                 144 non-null     float64
 62  Net Generation (MW) from Other Fuel Sources (Adjusted)         158957 non-null  float64
 63  Net Generation (MW) from Unknown Fuel Sources (Adjusted)       0 non-null       float64
 64  Region                                                         269426 non-null  object
dtypes: float64(59), int64(1), object(5)
memory usage: 133.6+ MB
```

[109]:
```
#check the range of main features:
check_list = ['Data Date','Hour Number','Demand (MW)','Demand (MW)','Demand (MW) (Adjusted)',␣
 ↪'Net Generation (MW)', 'Total Interchange (MW)' ,'Region']
for i in check_list:
    print(f"Field: {i}, Min value: {df_eia_930_balance[i].min()}, Max value:␣
 ↪{df_eia_930_balance[i].max()}")
```

```
Field: Data Date, Min value: 2024-07-01 00:00:00, Max value: 2024-12-31 00:00:00
Field: Hour Number, Min value: 1, Max value: 25
Field: Demand (MW), Min value: -992136.0, Max value: 9971121.0
Field: Demand (MW), Min value: -992136.0, Max value: 9971121.0
Field: Demand (MW) (Adjusted), Min value: 0.0, Max value: 153121.0
Field: Net Generation (MW), Min value: -992299.0, Max value: 9971318.0
Field: Total Interchange (MW), Min value: -32668.0, Max value: 12385.0
Field: Region, Min value: CAL, Max value: TEX
```

The detailed fields (such as Net Generation by different sources and imputed fields) have many missing values. The core fields such as Date, Time, Demand, Net Generation, Total Interchange, and Region are much more consistently

populated, but there are some values in some fields such as Demand, requiring further investigation and cleaning. Hour number 25 is questionable as well.

Option 3: EIA 861 M: This information is available to download in excel format. The below example is the 2025 sales and revenue data:

[26]:
```
df_861m_2025_sales = pd.read_excel("EIA/EIA 861 M/sales_ult_cust_2025.xlsx", skiprows=3,␣
 ↪header=None)
df_861m_2025_sales.columns = [
    "Year", "Month", "Utility_Number", "Utility_Name", "State", "Ownership", "Data_Status",
    "Residential_Revenue", "Residential_Sales", "Residential_Customers",
    "Commercial_Revenue", "Commercial_Sales", "Commercial_Customers",
    "Industrial_Revenue", "Industrial_Sales", "Industrial_Customers",
    "Transportation_Revenue", "Transportation_Sales", "Transportation_Customers",
    "Total_Revenue", "Total_Sales", "Total_Customers"
]
#At the end of the excel files for 861 M, there is one row of text that should be removed:
df_861m_2025_sales = df_861m_2025_sales[pd.to_numeric(df_861m_2025_sales["Year"],␣
 ↪errors="coerce").notna()]

#Data types for some fields need to be altered:
df_861m_2025_sales = df_861m_2025_sales.replace(".", None)
fields_to_convert = ["Residential_Revenue","Residential_Sales","Residential_Customers"
                     ,"Commercial_Revenue","Commercial_Sales","Commercial_Customers"
                     ,"Industrial_Revenue","Industrial_Sales","Industrial_Customers"
                    ]
df_861m_2025_sales[fields_to_convert] = df_861m_2025_sales[fields_to_convert].apply(pd.
 ↪to_numeric, errors="coerce")
df_861m_2025_sales.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 8913 entries, 0 to 8912
Data columns (total 22 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   Year                      8913 non-null   object
 1   Month                     8913 non-null   float64
 2   Utility_Number            8913 non-null   float64
 3   Utility_Name              8913 non-null   object
 4   State                     8913 non-null   object
 5   Ownership                 7677 non-null   object
 6   Data_Status               8913 non-null   object
 7   Residential_Revenue       8773 non-null   float64
 8   Residential_Sales         8771 non-null   float64
 9   Residential_Customers     8774 non-null   float64
 10  Commercial_Revenue        8911 non-null   float64
 11  Commercial_Sales          8911 non-null   float64
 12  Commercial_Customers      8912 non-null   float64
 13  Industrial_Revenue        8911 non-null   float64
 14  Industrial_Sales          8911 non-null   float64
 15  Industrial_Customers      8912 non-null   float64
 16  Transportation_Revenue    8913 non-null   float64
 17  Transportation_Sales      8913 non-null   float64
 18  Transportation_Customers  8913 non-null   float64
 19  Total_Revenue             8913 non-null   float64
 20  Total_Sales               8913 non-null   float64
 21  Total_Customers           8913 non-null   float64
```

```
dtypes: float64(17), object(5)
memory usage: 1.6+ MB
```

[107]:
```
#check the range of main features:
check_list = ['Year','Month','Total_Revenue','Total_Sales']
for i in check_list:
    print(f"Field: {i}, Min value: {df_861m_2025_sales[i].min()}, Max value:␣
 ↪{df_861m_2025_sales[i].max()}")

print(f"Unique states: {df_861m_2025_sales['State'].unique()}")
```

```
Field: Year, Min value: 2025, Max value: 2025
Field: Month, Min value: 1.0, Max value: 12.0
Field: Total_Revenue, Min value: 0.0, Max value: 58521316.408
Field: Total_Sales, Min value: 0.0, Max value: 407629714.809
Unique states: ['AK' 'AL' 'AR' 'AZ' 'CA' 'CO' 'CT' 'DC' 'DE' 'FL' 'GA' 'HI' 'IA'
'ID'
 'IL' 'IN' 'KS' 'KY' 'LA' 'MA' 'MD' 'ME' 'MI' 'MN' 'MO' 'MS' 'MT' 'NC'
 'ND' 'NE' 'NH' 'NJ' 'NM' 'NV' 'NY' 'OH' 'OK' 'OR' 'PA' 'RI' 'SC' 'SD'
 'TN' 'TX' 'UT' 'VA' 'VT' 'WA' 'WI' 'WV' 'WY' 'US']
```

Although useful demand information is available in the 861 M files, this file is aggregated at year-month level, and daily and hourly information is not available, making this dataset less suitable for our purposes given the inadequate level of granularity.

Weather Data (NOAA) The weather data is available up to 8/24/2025, but goes as far back as 1901. Files are provided in fixed-width format, and the station code at which the measurement was taken is included in the file name (it's not directly included in the file itself). Example:

[79]:
```
import pandas as pd

colspecs = [
    (0, 4),     # Year
    (5, 7),     # Month
    (8, 11),    # Day
    (11, 13),   # Hour
    (13, 19),   # Air Temperature
    (19, 25),   # Dew Point Temperature
    (25, 31),   # Sea Level Pressure
    (31, 37),   # Wind Direction
    (37, 43),   # Wind Speed
    (43, 49),   # Sky Condition
    (49, 55),   # Precipitation 1hr
    (55, 61),   # Precipitation 6hr
]

col_names = [
    "year", "month", "day", "hour",
    "air_temp", "dew_point", "sea_level_pressure",
    "wind_direction", "wind_speed", "sky_condition",
    "precip_1hr", "precip_6hr"
]

df = pd.read_fwf(
    "EIA/NOAA Weather/722140-93805-2024/722140-93805-2024",
    colspecs=colspecs,
    names=col_names
```

```
)

# Per spec, -9999 is basically a null:
df = df.replace(-9999, pd.NA)

for col in ["air_temp", "dew_point", "sea_level_pressure", "wind_speed", "precip_1hr",␣
 ↪"precip_6hr"]:
    df[col] = df[col].apply(pd.to_numeric, errors="coerce")
#Per spec, theres a scaling factor of 10:
for col in ["air_temp", "dew_point", "sea_level_pressure", "wind_speed", "precip_1hr",␣
 ↪"precip_6hr"]:
    df[col] = df[col] / 10

#Per spec, trace precipitation is denoted with -1. Replacing them with 0.00001:
df["precip_1hr"] = df["precip_1hr"].replace(-0.1, 0.0001)
df["precip_6hr"] = df["precip_6hr"].replace(-0.1, 0.0001)
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 8756 entries, 0 to 8755
Data columns (total 12 columns):
 #   Column              Non-Null Count  Dtype
---  ------              --------------  -----
 0   year                8756 non-null   int64
 1   month               8756 non-null   int64
 2   day                 8756 non-null   int64
 3   hour                8756 non-null   int64
 4   air_temp            8756 non-null   float64
 5   dew_point           8755 non-null   float64
 6   sea_level_pressure  8656 non-null   float64
 7   wind_direction      8514 non-null   object
 8   wind_speed          8744 non-null   float64
 9   sky_condition       6284 non-null   object
 10  precip_1hr          8036 non-null   float64
 11  precip_6hr          272 non-null    float64
dtypes: float64(6), int64(4), object(2)
memory usage: 821.0+ KB
```

[98]:
```
#check the range of main features:
check_list = ['year','month','day','hour', 'air_temp', 'sea_level_pressure', 'precip_1hr']
for i in check_list:
    print(f"Field: {i}, Min value: {df[i].min()}, Max value: {df[i].max()}")

print(f"Unique sky conditions: {df['sky_condition'].unique()}")
```

```
Field: year, Min value: 2024, Max value: 2024
Field: month, Min value: 1, Max value: 12
Field: day, Min value: 1, Max value: 31
Field: hour, Min value: 0, Max value: 23
Field: air_temp, Min value: -3.9, Max value: 37.8
Field: sea_level_pressure, Min value: 979.8, Max value: 1035.5
Field: precip_1hr, Min value: 0.0, Max value: 59.2
Unique sky conditions: [2 0 8 <NA> 6 4 9]
```

Main challenge with this source is the lack of location name in the files themselves. The station at which the measurement was taken is embedded in the file name (e.g. “722140” in file “722140-93805-2024” refers to “TALLAHASSEE REGIONAL AIRPORT”). Besides that, data seems suitable to be used as the main source for weather information for this project.

Which data source option should we go with? Given lack of daily and hourly data in option 3, it is the least suitable source for this project, since weather patterns can affect electricity demand by hours, if not minutes.

Both options 1 and 2 are suitable for this project.

Option 1 Advantages:

- Clean data (no missing values)
- Breakdown at Sub-region level

Option 1 Disadvantages:

- Net Generation and Adjusted values not included

Option 2 Advantages:

- Net Generation and Adjusted values included

Option 2 Disadvantages:

- Only available at Region level (not Subregion)
- Missing values require further research for cleaning (removal or imputation)
- Suspicious negative values for Demand require research and cleaning

Given the above, Option 1 seems to be the more suitable choice for our use case.

Task Assignments:

Task	Details	Lead	Support	Due
Check In 1 Steps 1-2	Problem Statement, Data Exploration, Task Breakdown	Behnam	Margaret, Matthew, Srinivas	4/7
Base Data Model	Design, Build, Load data, Validate via EDA	Margaret	Behnam, Matthew, Srinivas	4/13
Check In 2 Steps 3-4	Video presentation for steps 3-4	Matthew	Behnam, Margaret, Srinivas	4/19
Model 1	Design, Build, Train, Optimize, Execute, Validate	Margaret	Behnam, Matthew, Srinivas	4/22
Model 2	Design, Build, Train, Optimize, Execute, Validate	Behnam	Margaret, Matthew, Srinivas	4/22
Evaluation	Design, Build, Execute, Compile results	Srinivas	Behnam, Margaret, Matthew	4/24
Final Deliverables	Final Presentation	Matthew	Behnam, Margaret, Srinivas	5/3

Modeling Techniques

Model 1 - Regression: We selected regression as the approach for Model 1 because the target variable (hourly electricity demand) is continuous. Regression models are well suited for predicting continuous outcomes and provide a clear, interpretable framework for understanding how input features relate to the target. They also serve as a strong baseline against which more complex models (such as neural network) can be compared. We chose to implement two variants: linear and polynomial. This allows us to evaluate whether the relationship between weather and demand is adequately captured by a straight line, or whether curved terms are needed to improve predictions.

Model 1.1 - Linear Regression: Linear regression provides a simple and interpretable baseline. It allows us to measure whether the weather and time-based features have a meaningful linear relationship with demand in Virginia. The model is easy to implement using scikit-learn, has low computational requirements, and runs easily on a standard personal computer. Its main limitation is that it may underperform if the relationship between weather and demand is curved, since demand tends to be high at both low and high temperatures.

Model 1.2 - Polynomial Regression: To accommodate this expected non-linearity, the team also elected to run a polynomial regression. Electricity demand often responds to weather in a curved way, particularly during very hot or very cold periods. Polynomial regression keeps the same linear regression framework but expands the features

to include squared and higher-order terms, allowing the model to represent these non-linear effects. It can also be implemented in scikit-learn and should run on a standard computer, though its computational needs will be higher than linear regression because the polynomial expansion increases the number of input features. This additional complexity can improve fit, but it also increases the risk of overfitting and reduces interpretability. Regularization will be applied to mitigate this.

Model 2 - Neural Network: Model 2 uses a neural network, implemented in PyTorch. A neural network learns patterns through backpropagation and gradient descent and was selected for its ability to automatically learn non-linear relationships. This is relevant because non-linear relationships are expected in this problem. For example, the relationship between temperature and demand can be U-shaped (consumption increases during both heat and cold). Unlike a linear model, which requires manually engineered features to approximate such patterns, the neural network can learn them directly from the data. PyTorch was chosen for its fast computation and the flexibility it provides over network architecture and hyperparameters. The neural network also provides a meaningful comparison against Model 1's linear regression, allowing us to evaluate whether the added complexity of a neural network delivers better predictive performance.

The network architecture consists of an input layer, four hidden layers, and an output node. The input layer accepts weather features (temperature, dew point, sea level pressure, wind speed, precipitation) along with time-based features. The first hidden layer contains 64 neurons, the second contains 32 neurons, the third contains 16 neurons, and the fourth contains 8 neurons, all using ReLU activation. The output layer contains a single neuron predicting electricity demand.

Key hyperparameters include learning rate (0.001 using the Adam optimizer), batch size (32 or 64), and number of training epochs controlled by early stopping. Dropout rate and layer sizes may also be tuned depending on initial results.

Evaluation Plan

Our plan to evaluate model effectiveness uses a time-based train-test split: training on 2023 and 2024 data, and testing on 2025 data. Both the demand data and weather data come from the same respective sources (EIA 930 and NOAA ISD-Lite) across all three years. A time-based split was chosen over a random split because this is temporal data, and a random split would leak future patterns into the training set. The time-based approach simulates a realistic scenario where we are predicting future demand from historical patterns.

Evaluation Proposal 1 - Model 1 Regression Accuracy: Model 1 uses linear and polynomial regression. The primary evaluation measures predictive accuracy using two metrics:

R^2 measures how well the model fits the problem by quantifying the proportion of variance in demand explained by the model's predictions. An R^2 of 1.0 would mean the model perfectly explains all variation, while 0.0 would mean it performs no better than predicting the mean. Given that weather is not the only driver of electricity demand (economic activity, population, day of week, holidays, and other factors also play a role), we consider an R^2 above 0.60 a strong result.

Mean Squared Error (MSE) quantifies how far predictions deviate from actual values. It penalizes larger errors more heavily, giving us a clear measure of model accuracy with emphasis on avoiding big misses, which matters for grid reliability planning.

For the regression models specifically, we will compare linear versus polynomial fits to determine whether adding polynomial terms (such as squared temperature to capture the U-shaped demand relationship) meaningfully improves performance. Regularization will be applied to prevent overfitting, particularly for higher-degree polynomial terms that may fit noise in the training data. We will also examine the model coefficients to understand which weather features have the strongest influence on demand.

Evaluation Proposal 2 - Model 2 Neural Network Accuracy and Generalization: Model 2 uses a neural network implemented in PyTorch. Performance will be evaluated using the same R^2 and MSE metrics on the same 2025 test set, ensuring a direct comparison with Model 1.

Beyond raw accuracy, we will evaluate generalization by comparing training loss against test loss across epochs. If training loss continues to decrease while test loss plateaus or increases, the model is overfitting. Early stopping will halt training when validation loss stops improving, and dropout will be applied during training to prevent the network from memorizing patterns that don't generalize.

We will also evaluate the impact of hyperparameter tuning on performance. Starting with a learning rate of 0.001, batch size of 32, and the base architecture (64-32-16-8 hidden layers), we will experiment with different configurations and document how each change affects R^2 and MSE on the test set. This provides insight into how sensitive the neural network is to its configuration and whether the tuning effort is worthwhile compared to the simpler regression approach. All input features will be normalized before training, as neural networks are sensitive to differences in input scale.

Computational Feasibility: For this project to be practical, both models need to train and predict within a reasonable timeframe on standard hardware (a typical laptop without GPU acceleration). We will track and report training time for both models. The regression models are expected to train in seconds. The neural network should train within a few minutes at most. If either model requires more than 15 minutes to train, we would consider that a concern and look into reducing complexity. Prediction time (inference) should be near-instant for both approaches. We will report actual training times in our results to give a practical picture of the computational cost.

What Constitutes Success: We define success as follows: if either model achieves an R^2 above 0.60 on the 2025 test set, we consider the weather-based prediction approach viable for this problem. If the neural network outperforms the linear/polynomial regression by a meaningful margin (at least 5% improvement in R^2 or 10% reduction in MSE), we conclude that the added complexity is justified. If the simpler model performs comparably, we would recommend the regression approach for its interpretability and lower computational cost. Both models must also train within a reasonable timeframe on standard hardware to be considered practical for this use case.